

Язык программирования Go

Введение и основные конструкции

История создания

- 21 сентября 2007 года – начало проектирования (Кен Томпсон, Роб Пайк, Роберт Гризмер)
- Январь 2008 – начало разработки прототипа компилятора на С (Кен Томпсон)
- Май 2008 – начало разработки front-end для gcc (Ян Тейлор)
- Конец 2008 – к проекту присоединился Расс Кокс
- 11 ноября 2009 – первый релиз

Кен Томпсон, Роб Пайк, Роберт Гризмер, Ян Тейлор, Расс Кокс

Ключевые особенности (1)

- ❶ Простая система типов (отсутствие наследования, статический duck-typing).
- ❷ Поддержка замыканий (функции - полноправные данные).
- ❸ Поддержка параллельно выполняемых сопрограмм (горутин) и каналов.

Ключевые особенности (2)

- 4 Высокая скорость компиляции (пакеты, простая система типов).
- 5 Автоматическое управление памятью (сборка мусора).
- 6 Безопасность (отсутствие арифметики указателей, проверка границ).
- 7 Непротиворечивая грамматика.
- 8 Стандартизированное оформление кода (утилита `gofmt`).

Пример: Hello, World!

```
package main
import "fmt"
func main() {
    fmt.Printf("Hello, World!\n")
}
```

Компиляция и запуск:

```
go build hello.go
./hello
```

Что почитать / порешать

`(https://go.dev/tour/list`

`https://go.dev/doc/effective\_go,`
также `https://github.com/Ox0FACED/effective-go-ru/blob/main/`
`effective-go-ru.pdf`

`https://github.com/slon/shad-go`

`(https://github.com/go101/go101`

- Текст в Unicode. Регистр букв различается.
- Комментарии: `/* ... */` и `// ...`.
- Целочисленные константы и константы с плавающей точкой – как в C.
- Комплексные константы (например, `0.25i`).
- Символьные константы поддерживают Unicode через последовательности `\uxxxx` и `\Uxxxxxxxx`.

25 КЛЮЧЕВЫХ СЛОВ

break	default	func	interface	select
case	defer	go	map	struct
chan	else	goto	package	switch
const	fallthrough	if	range	type
continue	for	import	return	var

Predeclared identifiers

Константы:

`true false iota nil`

Типы:

`any comparable int int8 int16 int32 int64 uint uint8 uint16
uint32 uint64 uintptr float32 float64 complex128 complex64
bool byte rune string error`

Функции:

`min max make len cap new append clear copy close
delete complex real imag panic recover`

Идентификаторы можно переопределять:

```
var true = false
```

Exported identifiers

Регистр первой буквы определяет видимость имени за пределами пакета.

```
package example

const C0 = 9.8
const c1 = 15

var V0 string
var v1 string

func F0() {}
func f1() {}

type T0 int
type t1 string
```

Пакеты всегда называют в нижнем регистре и без подчеркиваний: `fmt`, `grpcmiddleware`

Интерпретируемые (в двойных кавычках):

```
"int main()\n" +  
"{\n" +  
"    printf(\"Hi!\\n\");\n" +  
"    return 0;\n" +  
"}\n"
```

Неинтерпретируемые (в обратных кавычках):

`

```
int main()  
{  
    printf ("Hi!\n");  
    return 0;  
}
```

`

Компилятор вставляет ; после непустой строки, если в конце стоит:

- ❶ идентификатор
- ❷ константа
- ❸ ключевое слово `break`, `continue`, `fallthrough` или `return`
- ❹ спецсимвол `++`, `--`, `)`, `]` или `}`

Пример: автоматическая вставка ;

```
package main
import "fmt"
const (
    ALPHA = iota
    BETA
    GAMMA
)
func main() {
    fmt.Printf("%d\n",
        BETA,
    )
}
```

```
package main;
import "fmt";
const (
    ALPHA = iota;
    BETA;
    GAMMA;
);
func main() {
    fmt.Printf("%d\n",
        BETA,
    );
};
```

4 типа объявлений: `var`, `const`, `type`, `func`

Пакет состоит из множества `.go` файлов.

Порядок объявлений в коде не важен.

Общая форма:

```
var name type = expression
```

Примеры:

```
var i, j, k int           // int, int, int
var b, f, s = true, 2.3, "four" // bool, float64, string
var f, err = os.Open(name) // os.Open returns a file and an error
```

short variable declaration

```
i, j := 0, 1
```

Существующим переменным присваиваются новые значения:

```
in, err := os.Open(infile)
// ...
out, err := os.Create(outfile)
```

Но должна объявляться хотя бы одна новая переменная:

```
f, err := os.Open(infile)
// ...
f, err := os.Create(outfile) // compile error: no new variables
```


Объявление переменных

Полная форма:

```
var x float32
var p, q, r *int
var a, b, c int = 1, 2, 3
```

Блок var:

```
var (
    x float32
    p, q, r *int
    a, b, c int = 1, 2, 3
)
```

Вывод типа:

```
var x = 0.1  
var a, b, c = 'Q', 2, 3.5
```

Сокращённая форма внутри функций (:=):

```
x := 0.1  
a, b, c := 'Q', 2, 3.5
```

Переменная, объявленная, но не инициализированная значением, получает "значение по умолчанию" — нулевое

- Числовые типы (и строковые литералы) — буквально ноль
- Строковый тип — пустая строка
- Указатели, структуры, функции... — `nil` (у каждого типа своё `nil`-значение)

Zero values — значения по умолчанию

Тип	Zero value
bool	false
int, int8...	0
uint, uint8...	0
float32, float64	0.0
complex64, complex128	0+0i
string	""
pointer (*T)	nil

Тип	Zero value
function	nil
interface	nil
slice	nil
map	nil
channel	nil
struct	все поля — zero values
array	все элементы — zero values

```
var (  
    a int           // 0  
    b string        // ""  
    c bool          // false  
    d *float64       // nil  
    e []int          // nil  
    f map[string]int // nil  
    g struct{ x int } // {0}  
)
```

Пустой идентификатор (`_`)

Функция может возвращать несколько значений.

```
a, b, c := f()    // Получаем все три значения  
a, _, c := f()   // Игнорируем второе
```

Пустой идентификатор не создает новую переменную.

Именованные константы

Объявление:

```
const PI float32 = 3.14
const A, B int = 10, 20
const x complex64 = 0.25i
```

Блок const:

```
const (
    PI float32 = 3.14
    A, B int = 10, 20
    x complex64 = 0.25i
)
```

Генератор констант `iota`

`iota` сбрасывается в 0 в начале каждого блока `const` и увеличивается на 1 для каждой следующей константы.

```
const (  
    a = iota    // a == 0  
    b = iota    // b == 1  
    c = iota    // c == 2  
)  
  
const (  
    x = 1 << iota // x == 1  
    y = 1 << iota // y == 2  
    z = 1 << iota // z == 4  
)
```

Опускание повторяющегося выражения

В блоке `const` можно опускать повторяющиеся выражения.

```
const (  
    A int16 = 'a' + iota  
    B  
    C  
)  
// A = 'a', B = 'b', C = 'c'
```


В Go используются **операторы** присваивания, инкремента и декремента.

- **Простое:** `a = b`
- **Составное:** `a += b`
- **Кортежное:** `a, b, c = f(x)`
- **Параллельное:** `a, b = b, a`
- **Инкремент/Декремент:** `a++`, `a--`

Оператор if

```
if простой_оператор; условие {  
    блок  
} else {  
    блок  
}
```

Пример:

```
if a, b := f(x); a < b {  
    return 0  
} else {  
    fmt.Printf("Hi")  
}
```

- Скобки вокруг условия не ставятся.
- Тело обязательно в {}.

Switch в Golang — основы

```
switch day {  
default: // выполняется последним  
    fmt.Println("Выходной")  
case "пн":  
    fmt.Println("Понедельник")  
case "вт", "ср", "чт":  
    fmt.Println("Середина недели")  
case "пт":  
    fmt.Println("Пятница")  
}
```

- **Нет** автоматического проваливания (fall through)
- Можно перечислять несколько значений через запятую
- default — необязателен
- Выполняется только первый подходящий case

```
switch char {  
case ' ', '\t', '\n', '\r':  
    fmt.Println("Пробельный символ")  
case '0', '1', '2', '3', '4', '5', '6', '7', '8', '9':  
    fmt.Println("Цифра")  
default:  
    fmt.Println("Другой символ")  
}
```

Switch без выражения — switch true

```
x := 42

if x < 0 {
    fmt.Println("Отрицательное")
} else if x == 0 {
    fmt.Println("Ноль")
} else if x < 10 {
    fmt.Println("Маленькое")
} else if x < 100 {
    fmt.Println("Среднее")
} else {
    fmt.Println("Большое")
}
```



```
x := 42

switch {
case x < 0:
    fmt.Println("Отрицательное")
case x == 0:
    fmt.Println("Ноль")
case x < 10:
    fmt.Println("Маленькое")
case x < 100:
    fmt.Println("Среднее")
default:
    fmt.Println("Большое")
}
```

```
switch x { ... }    // сравнивает x с case-выражениями
switch { ... }      // то же самое, что switch true { ... }
```

Switch с инициализацией

```
switch x := getValue(); {  
case x < 0:  
    fmt.Printf("x = %d - отрицательное\n", x)  
case x == 0:  
    fmt.Printf("x = %d - ноль\n", x)  
default:  
    fmt.Printf("x = %d - положительное\n", x)  
}  
// x здесь уже не видна
```

```
switch n := getNumber(); n % 2 {  
case 0:  
    fmt.Println(n, "чётное")  
case 1:  
    fmt.Println(n, "нечётное")  
}
```

```
switch file, err := os.Open(fname);  
↪ {  
case err != nil:  
    log.Fatal(err)  
default:  
    defer file.Close()  
    // работа с file  
}
```

fallthrough — принудительное проваливание

```
switch num {  
case 1:  
    fmt.Println("Один")  
    fallthrough  
case 2:  
    fmt.Println("Два")  
    fallthrough  
case 3:  
    fmt.Println("Три")  
default:  
    fmt.Println("Что-то ещё")  
}
```

Вывод для num = 1:

Один
Два
Три

Вывод для num = 2:

Два
Три

- fallthrough должен быть последним оператором в case
- Он безусловно передаёт управление в следующий case
- Используется редко

Type switch — переключение по типу

```
func describe(i interface{}) {  
    switch v := i.(type) {  
    case int:  
        fmt.Printf("Целое число: %d\n", v)  
    case string:  
        fmt.Printf("Строка: %q, длина %d\n", v, len(v))  
    case bool:  
        fmt.Printf("Булево: %v\n", v)  
    case []int:  
        fmt.Printf("Срез int'ов: %v\n", v)  
    case nil:  
        fmt.Println("Это nil")  
    default:  
        fmt.Printf("Неизвестный тип: %T\n", v)  
    }  
}
```

describe(42)	Целое число: 42
describe("hello")	Строка: "hello", длина 5
describe(true)	Булево: true
describe(nil)	Это nil

Сравнение if-else и switch

```
if err := process(); err != nil {  
    return err  
}
```

```
switch {  
case score >= 90:  
    grade = "A"  
case score >= 80:  
    grade = "B"  
default:  
    grade = "F"  
}
```


Классический пример

```
func f() bool {  
    return false  
}  
  
func isTrue() bool {  
    switch f() {  
    case true:  
        return true  
    default:  
        return false  
    }  
}
```

Результат: false

```
func f() bool {  
    return false  
}  
  
func isTrue() bool {  
    switch f() // компилятор вставит ;  
    { // компилятор вставит switch  
    case true:  
        return true  
    default:  
        return false  
    }  
}
```

Результат: true

Цикл for

Полная форма (как for C):

```
for простой_оператор; условие; простой_оператор {  
    блок  
}
```

Форма "while":

```
for условие {  
    блок  
}
```

Бесконечный цикл:

```
for {  
    блок  
}
```

break и continue работают как обычно.

Примеры циклов

Числа Фибоначчи:

```
for a, b := 1, 1; b < 100; a, b = b, a+b {  
    fmt.Println("%d\n", b)  
}
```

Быстрое возведение в степень:

```
res := 1  
for n > 0 {  
    if n & 1 == 1 {  
        res *= x  
    }  
    x *= x  
    n >>= 1  
}
```

Примитивные типы

Тип	Знак	Бит	Мин	Макс
int8	да	8	-128	127
uint8 (byte)	нет	8	0	255
int16	да	16	-32768	32767
uint16	нет	16	0	65535
int32 (rune)	да	32	-2^{31}	$2^{31} - 1$
uint32	нет	32	0	$2^{32} - 1$
int64	да	64	-2^{63}	$2^{63} - 1$
uint64	нет	64	0	$2^{64} - 1$
int	да	32/64	—	—
uint	нет	32/64	—	—
uintptr	нет	"дикий" указатель		
float32	да	32	$\pm 3.4 \cdot 10^{\pm 38}$ (7 цифр)	
float64	да	64	$\pm 1.7 \cdot 10^{\pm 308}$ (15 цифр)	
complex64	(float32, float32)			
complex128	(float64, float64)			

```
var A [10]int
const N = 100
var B, C, D [N]float64
var M [5][4]int    // эквивалентно [5]([4]int)
```

- Индексация: `A[i]`
- Выход за границы запрещен.
- Длина: `len(A)`

range для перебора массива

```
package main

import "fmt"

func main() {
    var A [10]int
    for i := 0; i < 10; i++ {
        fmt.Scanf("%d", &A[i])
    }
    for i, x := range A {
        fmt.Printf("A[%d] = %d\n", i, x)
    }
}
```

Создание и инициализация безымянного массива:

```
[10]int{1, 2, 3}           // остальные - нули  
[...]int{1, 2, 3}         // массив размера 3
```

Пример передачи массива

```
package main

import "fmt"

func f(a [5]int) {
    for i, x := range a {
        fmt.Printf("%d) %d; ", i, x)
    }
}

func main() {
    f([...]int{10, 20, 30, 40, 50})
}
```

Вывод: 0) 10; 1) 20; 2) 30; 3) 40; 4) 50;


```
var p *int
var q **int
var ap *[10]int
```

- Разыменование: *p
- Взятие адреса: &(A[i])
- Арифметика указателей **запрещена**.
- Индексация указателей **запрещена**.
- void*-указателей **нет**.

Массивы в Go используются редко из-за фиксированного размера.

Срез — это структура данных, описывающая фрагмент массива:

- указатель на первый элемент фрагмента;
- длина (количество элементов);
- ёмкость (capacity).

```
var A []int
var P []*int
var M [][]float64
```

- К срезам применима операция индексации `[]`.
- Выход за границы **запрещен**.

Срезы (slices) - окно в подлежащий массив

len=4, cap=8
slice := array[2:6]



Подлежащий массив (capacity = 10)
[10]int{0,1,2,3,4,5,6,7,8,9}

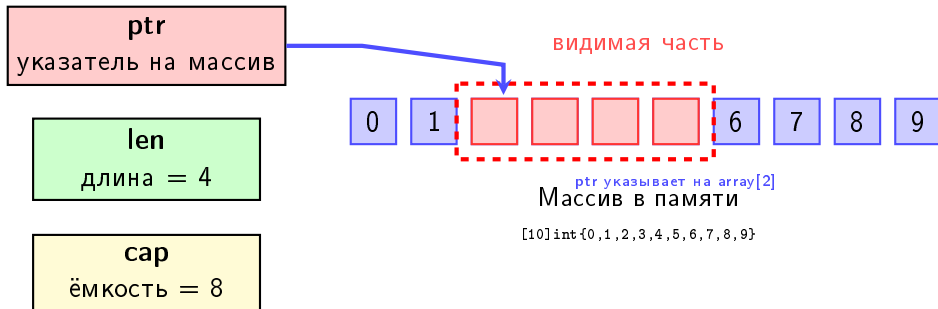
len=5, cap=6
slice2 := slice[2:7]



```
array :=  
↪ [10]int{0,1,2,3,4,5,6,7,8,9}  
slice := array[2:6]  
// len=4, cap=8
```

```
slice2 := slice[2:7]  
// len=5, cap=6  
  
slice3 := array[:8]  
// len=8, cap=10
```

Внутреннее устройство среза

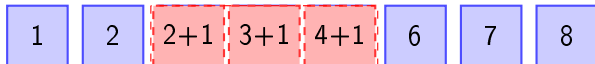


```
struct { ptr *array len int cap int }
```

- **ptr** указывает на первый элемент среза (не на начало массива!)
- **len** — количество видимых элементов
- **cap** = len(массива) - (ptr - начало массива)
- Несколько срезов могут ссылаться на один массив

Изменение массива через срез

`slice := array[2:5]`



После `slice[1] = 99`:



- Срез не копирует данные — он показывает окно в массив
- Изменение через срез меняет исходный массив
- Другие срезы, показывающие ту же область, тоже увидят изменения

```
array := [8]int{1,2,3,4,5,6,7,8}
slice := array[2:5] // [3,4,5]

slice[1] = 99      // меняем элемент среза

fmt.Println(array) // [1,2,3,99,5,6,7,8] - массив изменился!
fmt.Println(slice) // [3,99,5] - срез показывает новое значение
```

append и переполнение ёмкости

1. Исходный срез (cap=4):

`s := []int{10,20,30} (len=3, cap=4)`



2. Append (в пределах cap):

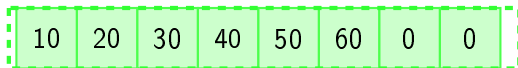
`s = append(s, 40) (len=4, cap=4)`



3. Append (превышение cap) — создаётся новый массив:



`s = append(s, 50, 60) (len=6, cap=8)`



```
s := make([]int, 3, 4) // len=3, cap=4
s[0], s[1], s[2] = 10, 20, 30

s = append(s, 40)      // len=4, cap=4 (ещё ещё тот же массив)
s = append(s, 50, 60)  // cap превышена - новый массив (len=6, cap=8)
```


len и cap для срезов

- len — количество "занятых" элементов.
- cap — общая вместимость.

```
package main

import "fmt"

func main() {
    var a []int
    fmt.Printf("%d %d\n", len(a), cap(a))
}
```

Вывод: 0 0

Составной литерал среза

```
package main

import "fmt"

func main() {
    a := []int{1, 2, 3}
    fmt.Printf("%d %d\n", len(a), cap(a))
}
```

Вывод: 3 3

Операция вырезания

```
package main
import "fmt"
func main() {
    array := [7]byte{'a', 'b', 'c', 'd', 'e', 'f', 'g'}
    slice := array[2:5] // c, d, e
    for _, x := range slice {
        fmt.Printf("%c ", x)
    }
}
```

Вывод: c d e

Значения по умолчанию в срезах

```
array := [7]byte{'a', 'b', 'c', 'd', 'e', 'f', 'g'}
```

```
array[:4] []byte{'a', 'b', 'c', 'd'}
```

```
array[2:] []byte{'c', 'd', 'e', 'f', 'g'}
```

```
array[:] []byte{'a', 'b', 'c', 'd', 'e', 'f', 'g'}
```

Срез – это окно в массив

Изменение среза меняет исходный массив.

```
package main
import "fmt"
func main() {
    A := [5]int{1, 2, 3, 4, 5}
    b := A[2:] // b ссылается на A[2], A[3], A[4]
    b[1] = 10  // меняет A[3]
    for _, x := range A {
        fmt.Printf("%d ", x)
    }
}
```

Вывод: 1 2 3 10 5

`cap(slice) = len(array) - start`

```
package main
import "fmt"
func main() {
    var A [100]int
    slice := A[10:20]
    fmt.Printf("%d %d", len(slice), cap(slice))
}
```

Вывод: 10 90

```
package main
import "fmt"
func main() {
    var A [100]int
    slice := A[10:20]    // len=10, cap=90
    slice2 := slice[5:]  // от индекса 5 (всего 5 элементов)
    fmt.Printf("%d %d", len(slice2), cap(slice2))
}
```

Вывод: 5 85 (cap = 90 - 5)

Расширение среза (в пределах ёмкости)

```
package main
import "fmt"
func main() {
    var A [100]int
    slice := A[10:20]    // len=10, cap=90
    slice2 := slice[:30] // увеличиваем длину slice2 до 30
    fmt.Printf("%d %d", len(slice2), cap(slice2))
}
```

Вывод: 30 90

Копирование срезов: copy

```
package main
import "fmt"
func main() {
    A := [8]int{0, 1, 2, 3, 4, 5, 6, 7}
    copy(A[1:3], A[5:]) // копирует A[5], A[6] в A[1], A[2]
    for _, x := range A {
        fmt.Printf("%d ", x)
    }
}
```

Вывод: 0 5 6 3 4 5 6 7

Создание срезов: make

```
A := make([]int, length, capacity)
```

Создаёт в куче массив размера `capacity` и возвращает срез длины `length` на него.

```
A := make([]int, 0, 100) // длина 0, ёмкость 100
```

Сокращённая форма make

```
A := make([]int, capacity)
```

Длина и ёмкость будут равны capacity.

```
A := make([]int, 100) // длина 100, ёмкость 100
```

Динамическое расширение: `append`

```
slice = append(slice, x1, x2, ...)
```

Добавляет элементы в конец среза. При нехватке ёмкости выделяет новый массив.

Пример append

```
package main

import "fmt"

func main() {
    s := make([]int, 0, 5)
    s = append(s, 1)
    for a, b := 1, 1; b < 100; a, b = b, a+b {
        s = append(s, b)
    }
    for _, x := range s {
        fmt.Printf("%d ", x)
    }
}
```

Вывод: 1 1 2 3 5 8 13 21 34 55 89

- Встроенный тип `string`.
- Неизменяемые массивы `byte` (UTF-8).
- Индексация: доступ к **байтам** (`s[i]`). Запись запрещена.
- Вырезание: `s[2:5]` возвращает **строку** (байты).
- Конкатенация: `s + t` (создаёт новую строку).

`len` возвращает количество **байт**.

```
package main
import "fmt"
func main() {
    s := "кошка"
    fmt.Printf("len(%q) == %d\n", s, len(s))
}
```

Вывод: `len("кошка") == 10`

Операция: $T(x)$

- `string(1072) → "a"` (кодировка точка)
- `string([]byte{...}) → строка из байт`
- `[]byte(s) → срез байт (копия)`
- `string([]rune{...}) → строка из рун`
- `[]rune(s) → срез рун (копия)`

Пример: строка в срез рун

```
package main
import "fmt"
func main() {
    s := "кошка"
    a := ([]rune)(s) // или []rune(s)
    for _, x := range a {
        fmt.Printf("%c - %d\n", x, x)
    }
}
```

к - 1082

о - 1086

ш - 1096

к - 1082

а - 1072

Объявление функций

```
func имя(параметры) результат {  
    блок  
}
```

- Параметры: (p1 тип1, p2 тип2, ...)
- Можно сокращать: (a, b int, c string)
- Возвращаемые значения указываются как параметры или просто тип.

Именованные возвращаемые значения

```
func minmax(a, b int) (min, max int) {  
    if a < b {  
        min, max = a, b  
    } else {  
        max, min = a, b  
    }  
    return // "золый" return возвращает min, max  
}
```

```
func min(a, b int) int {  
    if a < b {  
        return a  
    }  
    return b  
}
```

Неименованные возвращаемые значения

```
func minmax(a, b int) (int, int) {  
    if a < b {  
        return a, b  
    }  
    return b, a  
}
```

Если функция ничего не возвращает:

```
func hello(name string) {  
    fmt.Printf("Hello, %s!\n", name)  
}
```

Композиция функций

```
func g(x int) (int, int) { return x, -x }  
  
func f(a, b int) int { return a + b }  
  
func main() {  
    fmt.Printf("%d", f(g(5))) // f(5, -5) = 0  
}
```

Вывод: 0

Функции с переменным числом аргументов

```
func concat(s ...string) string {  
    length := 0  
    for _, x := range s { length += len(x) }  
    result, pos := make([]byte, length), 0  
    for _, x := range s {  
        copy(result[pos:], ([]byte)(x))  
        pos += len(x)  
    }  
    return string(result)  
}  
  
func main() {  
    fmt.Print(concat("alpha", "beta", "gamma"))  
}
```

Передача среза в ...

```
func main() {  
    a := []string{"alpha", "beta", "gamma"}  
    fmt.Print(concat(a...))  
}
```

Функции как данные

```
var f func(a, b int)
var g func(x float32) string
var A []func([]byte) (int, int)
```

Тип функции: `func` (сигнатура).


```
func main() {  
    var x int  
    f := func(y int) int { return x + y }  
    x = 5  
    fmt.Printf("%d", f(10))  
}
```

Вывод: 15

От стрелочной нотации к синтаксису Go

Стрелочная нотация:

```
int -> string
(int, string) -> bool
(int) -> (string, error)
(int -> bool) -> float64
int -> (string -> []byte)
```

Синтаксис Go:

```
func(int) string
func(int, string) bool
func(int) (string, error)
func(func(int) bool) float64
func(int) func(string) []byte
```

Стираем стрелки — получаем Go! Стрелочная нотация → заменяется на `func(...)` ...

Правоассоциативность функциональных типов

Стрелочная нотация:

`int -> string -> []byte`

`int -> (string -> []byte)`
└───────────┘
|
v
функция

Синтаксис Go:

```
func(int) func(string) []byte
```

`func(int) (func(string) []byte)`
└───────────┘
|
v
функция

```
type StringProcessor func(int) func(string) []byte

// Функция, возвращающая функцию
func makePrefixer(prefix string) func(string) string {
    return func(s string) string {
        return prefix + s
    }
}
```

Функции, возвращающие кортежи (несколько значений)

Стрелочная нотация:

(string, int) ->
(string, error)

```
func(string, int) (string, error)
```

```
func(string, int) (string, error)
      \_____/ \_____/
      параметры возвращаемые
                        значения
```

```
func divide(a, b float64) (float64, error) {
    if b == 0 {
        return 0, fmt.Errorf("division by zero")
    }
    return a / b, nil
}
```

Пример: обобщённая сортировка

```
func sort(n int, less func(int, int) bool, swap func(int, int)) {  
    for j := n-1; j > 0; j-- {  
        k := j  
        for i := j-1; i >= 0; i-- {  
            if less(k, i) { k = i }  
        }  
        swap(j, k)  
    }  
}  
  
func main() {  
    A := []int{5, 2, 6, 1, 3, 5, 0, 9}  
    sort(len(A),  
        func(i, j int) bool { return A[i] < A[j] },  
        func(i, j int) { A[i], A[j] = A[j], A[i] },  
    )  
    for _, x := range A { fmt.Printf("%d ", x) }  
}
```

Рекурсивные замыкания

```
func main() {  
    var fact func(int) int  
    fact = func(x int) int {  
        if x == 0 {  
            return 1  
        }  
        return x * fact(x-1)  
    }  
    fmt.Printf("%d", fact(4))  
}
```